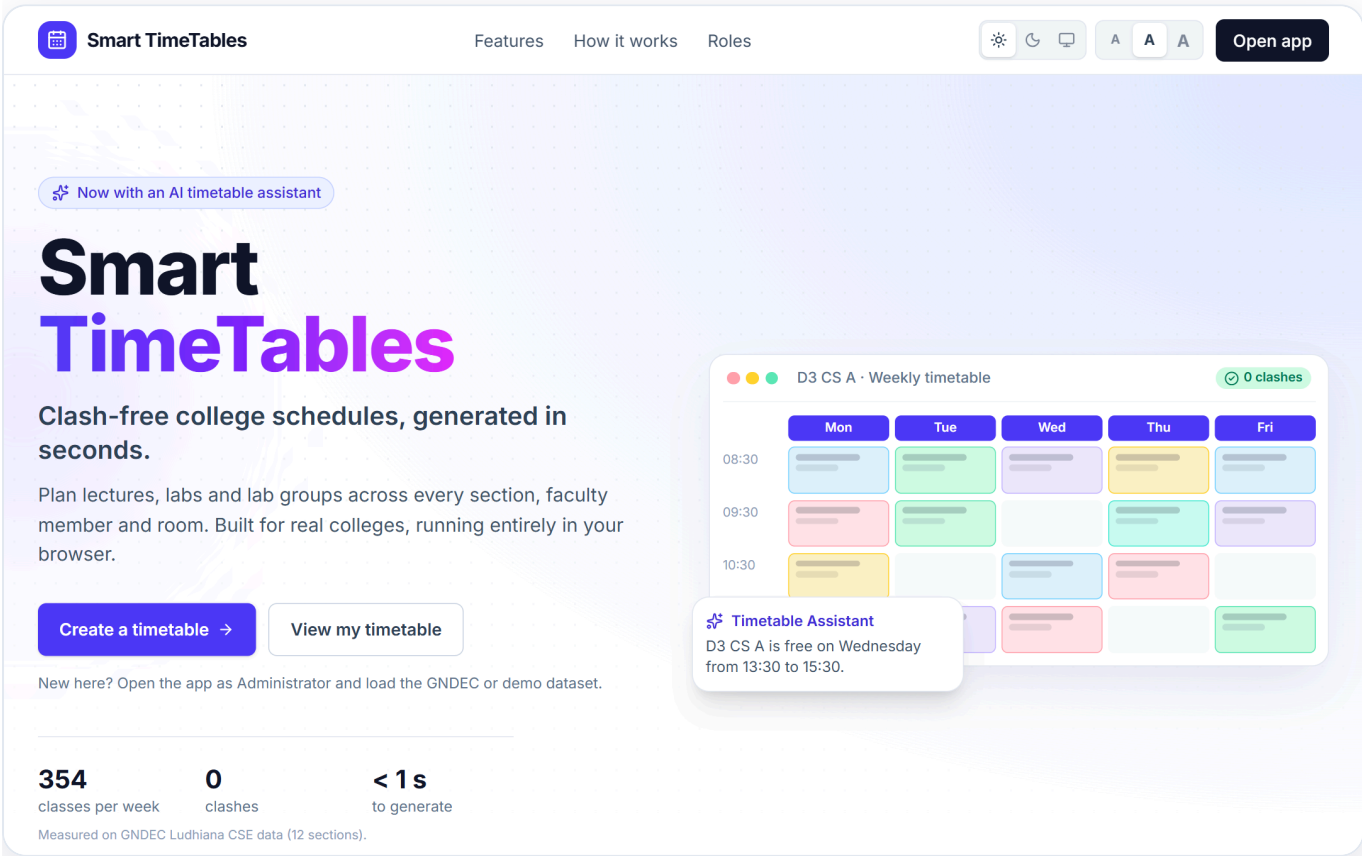
Hackathon Progress Report · 21 October 2026

Smart TimeTables

A clash-free college timetable generator that runs entirely in the browser, tested on real data from GNDEC Ludhiana (CSE department).

354	0	~0.5 s	18 / 18
classes scheduled per week (GNDEC CSE)	clashes across 20 test runs	average generation time	automated tests passing



Project	Smart TimeTables	**Institution**	GNDEC Ludhiana
Team / Name		**Roll No.**	
Stack	React · Vite · Claude API	**Status**	Working prototype

01 Executive summary

Making a college timetable by hand takes days and usually ends with clashes: a teacher booked in two rooms, a lab split across lunch, or two sections in one classroom. **Smart TimeTables** generates a complete weekly timetable for every section, teacher and room in about a second, guarantees zero clashes, and gives each role (administrator, HOD, faculty, student) its own view.

The app works for **any college**: the administrator enters the time structure, rooms, faculty, subjects and sections through a guided wizard. For a realistic test, it ships with a dataset built from **GNDEC Ludhiana's public CSE documents** for July–December 2026: 12 sections, 51 faculty, 19 rooms and labs, and 354 classes a week, including labs split into half-groups (A1/A2) as GNDEC actually runs them.

12 sections (D2 and D3 CS A–F)	**51** faculty from 5 departments	**19** classrooms and labs
94 / 100 average quality score	**83%** of lab groups scheduled side by side	**12** tested, committed build steps

What is complete

Area	Delivered	Status
Role-based app	Landing page with Administrator, HOD/Coordinator, Faculty and Student workspaces	Done
Setup wizard	Time slots with real clock times, rooms, subjects, faculty (with unavailable slots), sections	Done
Scheduling engine	Backtracking search with hard constraints, soft preferences, pre-check and plain-language failure reasons	Done
Real college data	GNDEC Ludhiana CSE dataset with lab groups, multiple lab sessions and separate lab teachers	Done
Timetable views	By section, teacher and room; HOD department filter; faculty year filter; student lab-group filter	Done
Design and access	Professional UI, light/dark theme, three text sizes, mobile layout, print to A4	Done
AI assistant	Claude-powered chat that answers questions about the loaded timetable	Built
Quality	18 automated tests, independent clash checker, benchmarked on 20 runs	Done
Deployment	Ready for Vercel; AI assistant needs the API key configured	Next

02 Problem and objectives

The problem

- Timetables are built manually in spreadsheets, often over several days each semester.
- Clashes are easy to miss: one teacher in two places, a room double-booked, a section with two classes at once.
- Labs need consecutive periods, a lab room big enough, and are often taken in half-groups.
- Faculty unavailability and workload limits must be respected.
- Students and faculty need a clear, mobile-friendly way to see their own schedule.

Objectives

- Work for **any college**, with no hard-coded structure.
- Generate a **clash-free** timetable in seconds, or explain clearly why it is impossible.
- Run **entirely in the browser** (no database or login), deployable as a static site.
- Give each role its own simple view, readable on phones and printable.
- Prove it on **real data** from our own college.

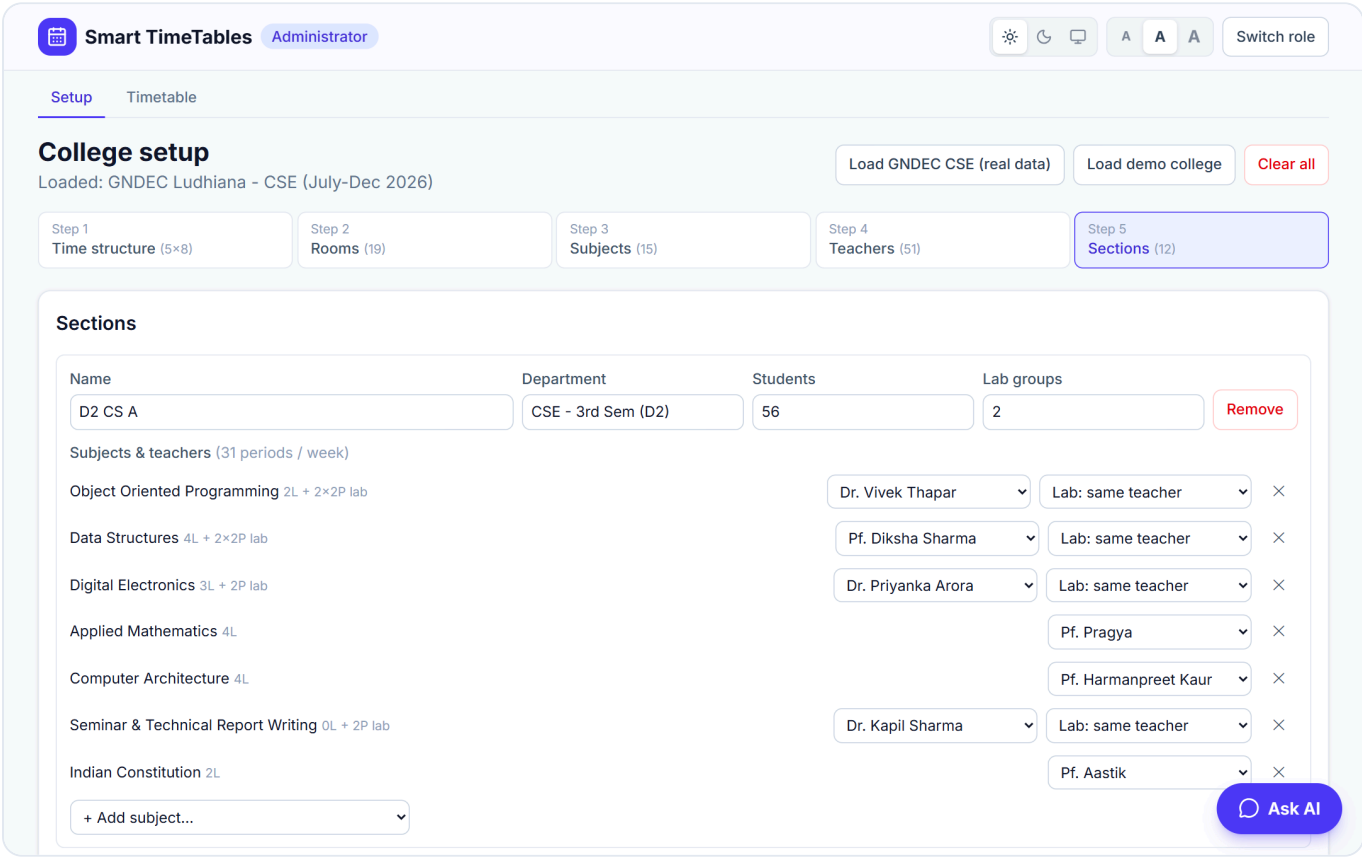
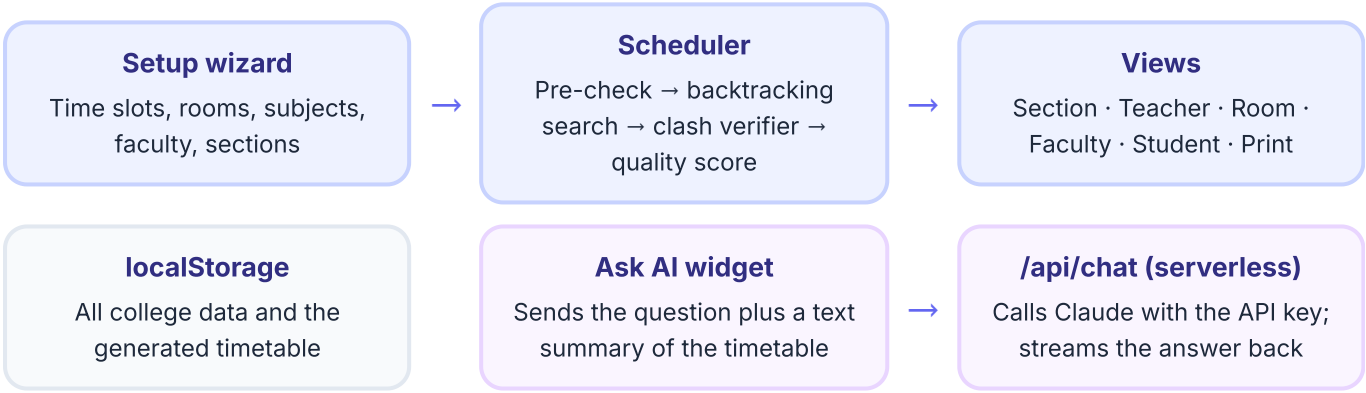


Figure 1. The administrator setup wizard (Sections step) with the GNDEC data loaded: D2 CS A with 2 lab groups, its subjects, lecture and lab teachers.

03 Technology and architecture

Layer	Technology	Why
Frontend	React 18, Vite 5	Fast development, small production bundle
Styling	Tailwind CSS v4, Inter font, Lucide icons	Consistent design system; dark mode by flipping the colour scale
Scheduling engine	Plain JavaScript module (`src/scheduler.js`)	No framework dependency; runs in the browser and in tests
Storage	Browser localStorage	No backend needed; data stays on the user's device
AI assistant	Claude API (`claude-opus-5-5`) via a serverless function	Natural-language questions about the timetable; API key kept on the server
Testing	Vitest	Same configuration as the build; 18 tests
Hosting	Vercel	Static site plus one serverless function, no configuration file



Codebase: about 3,500 lines across the frontend, scheduler, serverless function and tests.

04 Features

Administrator

- 5-step setup wizard: **Time structure** → **Rooms** → **Subjects** → **Teachers** → **Sections**.
- Real clock times: day start, period length, lunch position and length (e.g. 08:30–17:30, lunch 12:30).
- Rooms with capacity and type (classroom, lab, seminar hall).
- Faculty with department, subjects they can teach, maximum hours, and a clickable grid of unavailable slots.
- Subjects with lectures per week, lab block length and lab sessions per week.
- Sections with student count, number of lab groups, and a teacher (plus optional separate lab teacher) per subject.
- One-click loaders for the GNDEC CSE dataset and a demo college.
- Generate button with summary: classes placed, clashes, quality score, time taken.

HOD / Coordinator

- Timetable by **section**, **teacher** or **room**.
- Department filter.
- Live clash count and quality breakdown (repeated subjects, long teaching runs, gaps).

Faculty

- Pick your name to see your weekly schedule.
- **Year / class filter** (e.g. D2 or D3) to see one batch at a time.
- Shows periods, sections and total load against the maximum.

Student

- Pick department, section and **lab group** (e.g. C2) to see only your own labs.
- Phone layout with one card per day; choices are remembered.

Timetable design

- Days across the top, real time slots down the side, lunch row.
- One colour per subject with accent bar, subject code, teacher and room icons.
- Lab badges with group name; parallel lab groups shown side by side.
- Today's column highlighted; subject legend with weekly periods.
- **Print** button: one clean A4 landscape page per timetable.

Experience and accessibility

- Professional landing page with product preview, roles, features and "how it works".
- **Light, dark or system** theme.
- **Three text sizes** (16, 18, 20 px base) for readability.
- Full-width layout on desktop, responsive on phones.
- **Ask AI** assistant on every screen with suggested questions.

05 The scheduling engine

The core of the project is `src/scheduler.js`, a framework-free module that turns the college's requirements into a timetable.

Hard constraints (never broken)

- No teacher, room or section is double-booked.
- Room capacity is enough for the section (or lab group) and the room type matches (labs in lab rooms).
- Lab periods are consecutive and never cross the lunch break.
- Faculty unavailable slots are respected.
- A lecture blocks the whole section; a group lab blocks only that group, so A1 and A2 can be in two labs at once.

Soft preferences (scored)

- Spread each subject across different days.
- Avoid more than 3 back-to-back periods for a teacher.
- Balance each teacher's day.
- Run a section's lab groups exactly side by side.
- Prefer snug rooms and earlier periods; keep the seminar hall free.

How it works

#	Step	What happens
1	Pre-check	Detects impossible input before searching: teacher over maximum hours, section needing more periods than the week has, no room or lab big enough, lab longer than the gap between breaks, not enough lab blocks in the week. Each problem is reported in plain English.
2	Break into units	Every lecture hour becomes a 1-period unit; every lab session for every lab group becomes one block of consecutive periods needing a lab room.
3	Candidate slots	For each unit, all valid (day, start period, room) options are precomputed.
4	Most-constrained first	At each step the unit with the fewest remaining options is placed next, so hard-to-place labs go first.
5	Backtracking with busy grids	Teacher, room and section-group grids make every clash check instant. Options are tried best-score first; dead ends are undone.
6	Restarts and time limit	A step limit, random restarts and a 4-second budget keep the app responsive even for impossible inputs.
7	Verify and score	An independent checker re-verifies every hard constraint; a 0–100 quality score summarises the soft preferences.

> **Example failure message** (when the input cannot fit):
> *"Couldn't place Fluid Mechanics lab for ME-A (best attempt left 106 classes unplaced). Of its 16 possible time slots, 16 have every suitable lab taken. Try adding a lab, giving Prof. Neha Gupta more free slots, or reducing hours."*

06 Real data: GNDEC Ludhiana (CSE)

To test on a real college, a dataset was built from documents the CSE department publishes on `cse.gndec.ac.in`:

Source document	Used for
Section-wise subject allotment, July–Dec 2026	Subjects and the teacher for every section A–F of B.Tech 3rd (D2) and 5th (D3) semester, including separate lab teachers
Published departmental timetable (w.e.f. 19/08/2026)	Day structure (Mon–Fri, 08:30–17:30, lunch 12:30–13:30), weekly lectures and labs per subject, labs in half-groups A1/A2, rooms in use
Classrooms and laboratories infrastructure lists	Room and lab seating capacities

What the dataset contains

- 12 sections, 56 students each, 2 lab groups each
- 15 subjects; OOP and Data Structures have 2 lab sessions a week
- 51 faculty from CSE, Applied Sciences, MPE and Civil
- 10 classrooms (75 seats) and 9 labs (25–35 seats)
- 354 classes per week to schedule

Simplifications

- Tutorials counted as lectures.
- 5th-semester electives (which mix sections) left out.
- One common lunch slot (12:30) for both years.
- DBMS and Hardware lab capacities not published; 30 assumed.

> The app labels every GNDEC timetable as **"Generated by this app, not the official GNDEC timetable"**. Student roll lists in the source documents were not used.

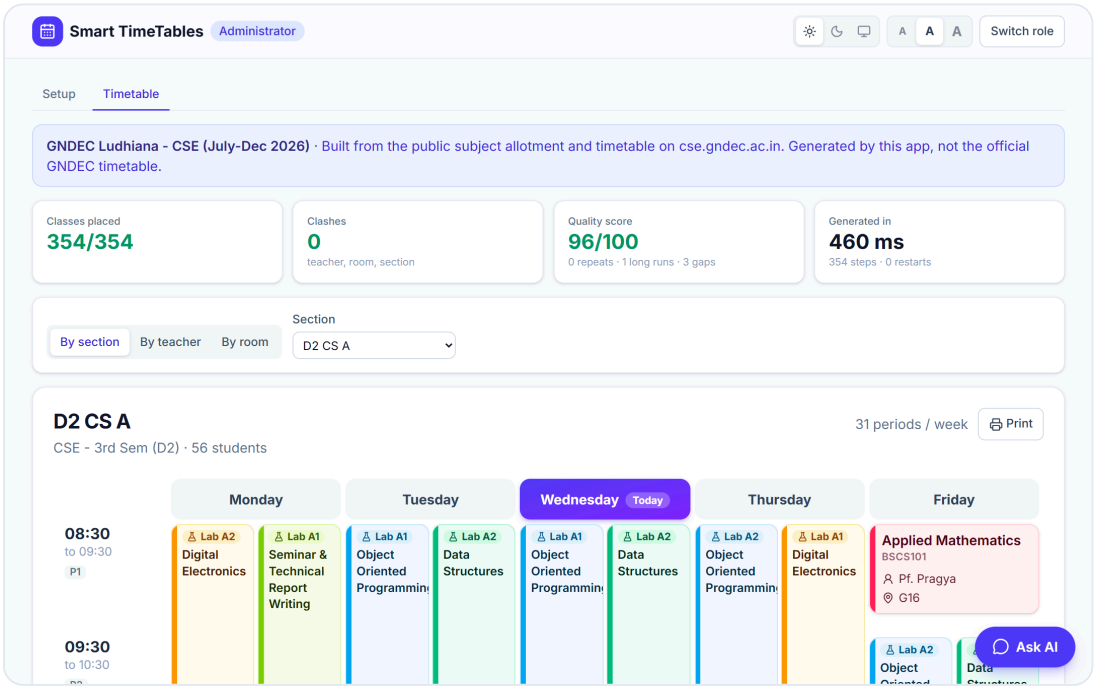


Figure 2. GNDEC data generated: 354/354 classes placed, 0 clashes, quality score and generation time.

07 Screenshots



Figure 3. Section timetable for D2 CS A. Labs for groups A1 and A2 run side by side in different labs; lunch row, subject legend and Print button.

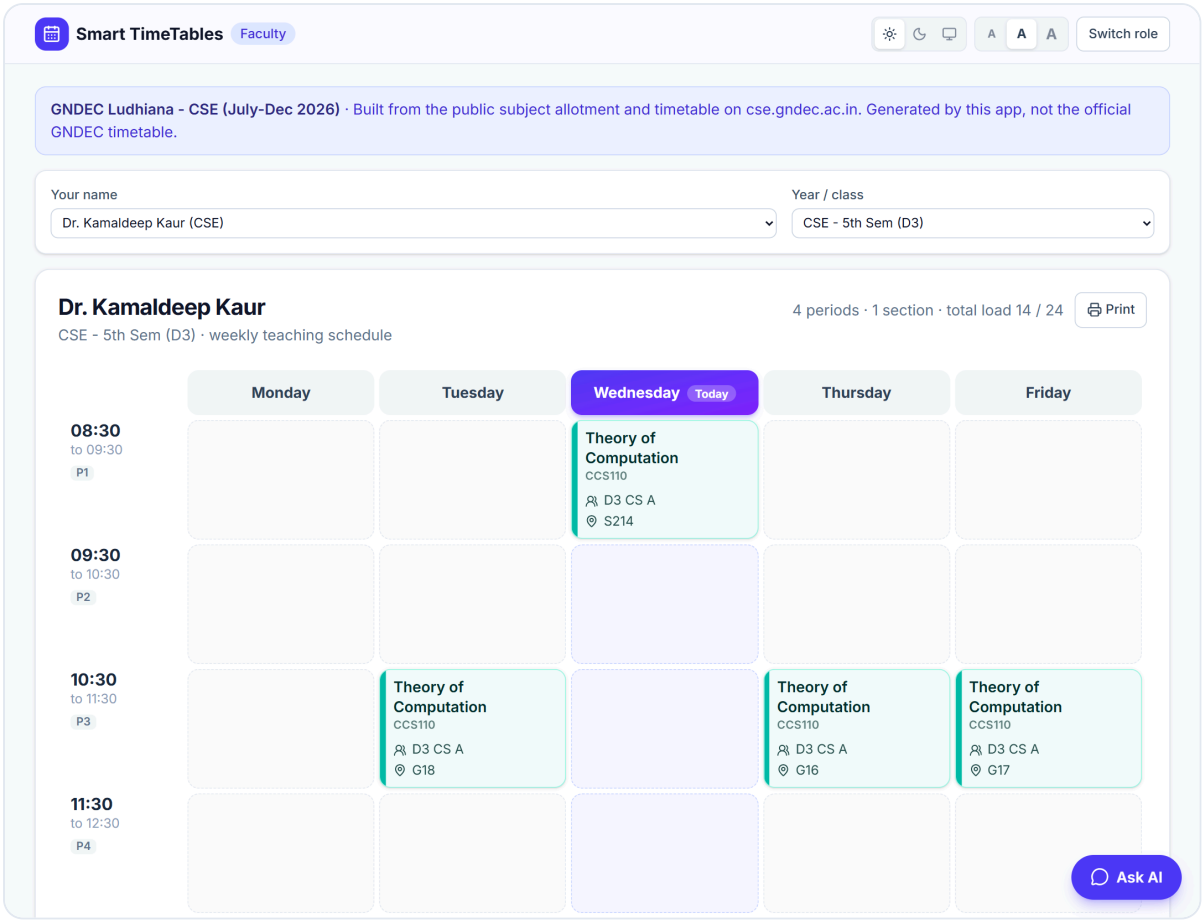


Figure 4. Faculty view with the Year / class filter set to D3: only this teacher's 5th-semester classes, with total load shown.

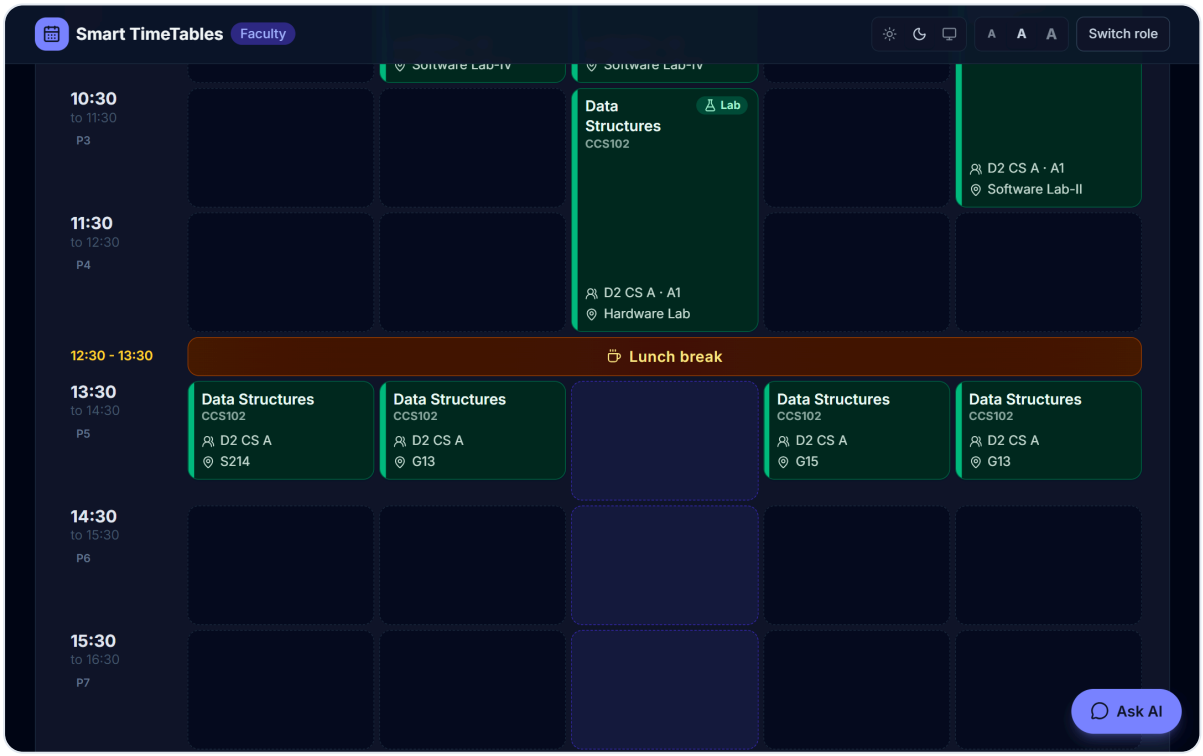


Figure 5. Dark theme, with the text size and theme switches in the header.

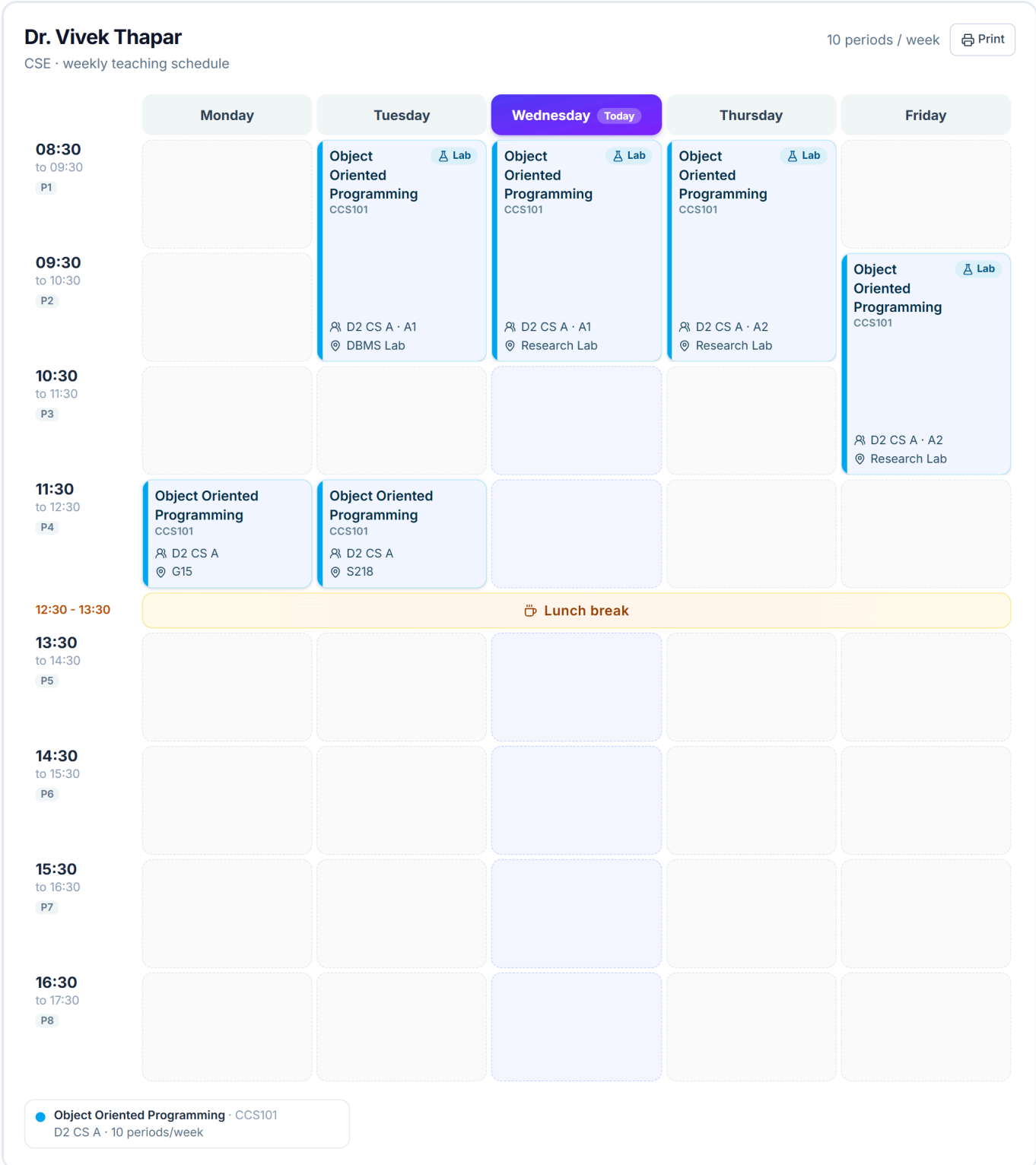


Figure 6. HOD/Admin "By teacher" view: each class shows the section, lab group and room.

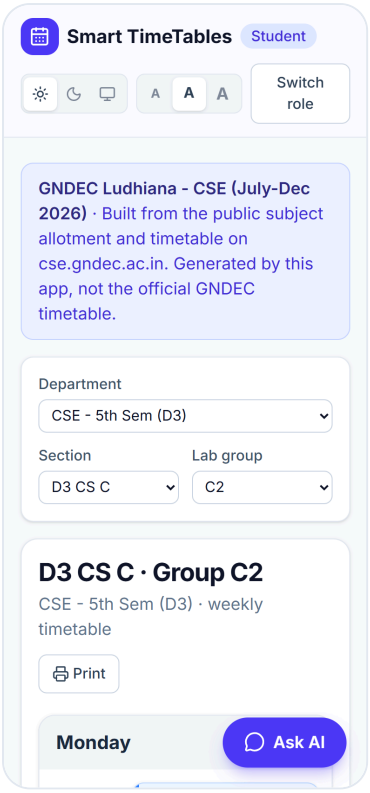


Figure 7. Student view on a phone: department, section and lab group C2.

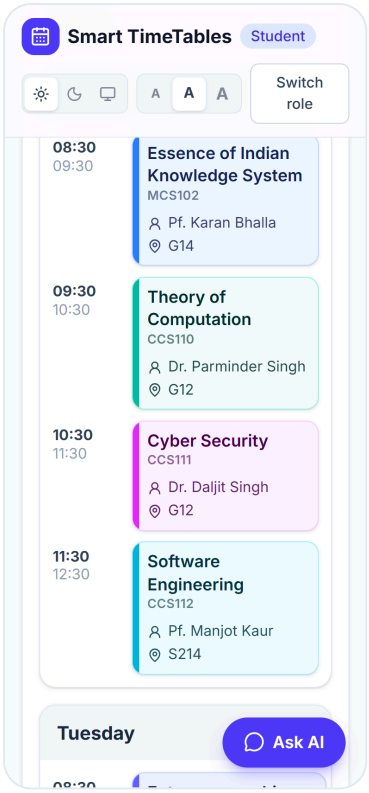


Figure 8. Day cards on mobile with times and class details.

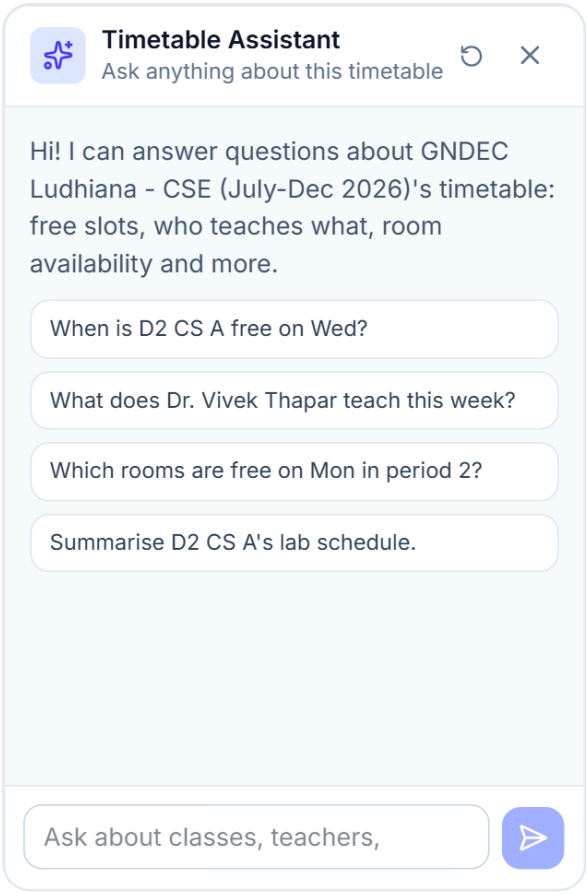


Figure 9. "Ask AI" assistant with suggested questions about the loaded timetable.

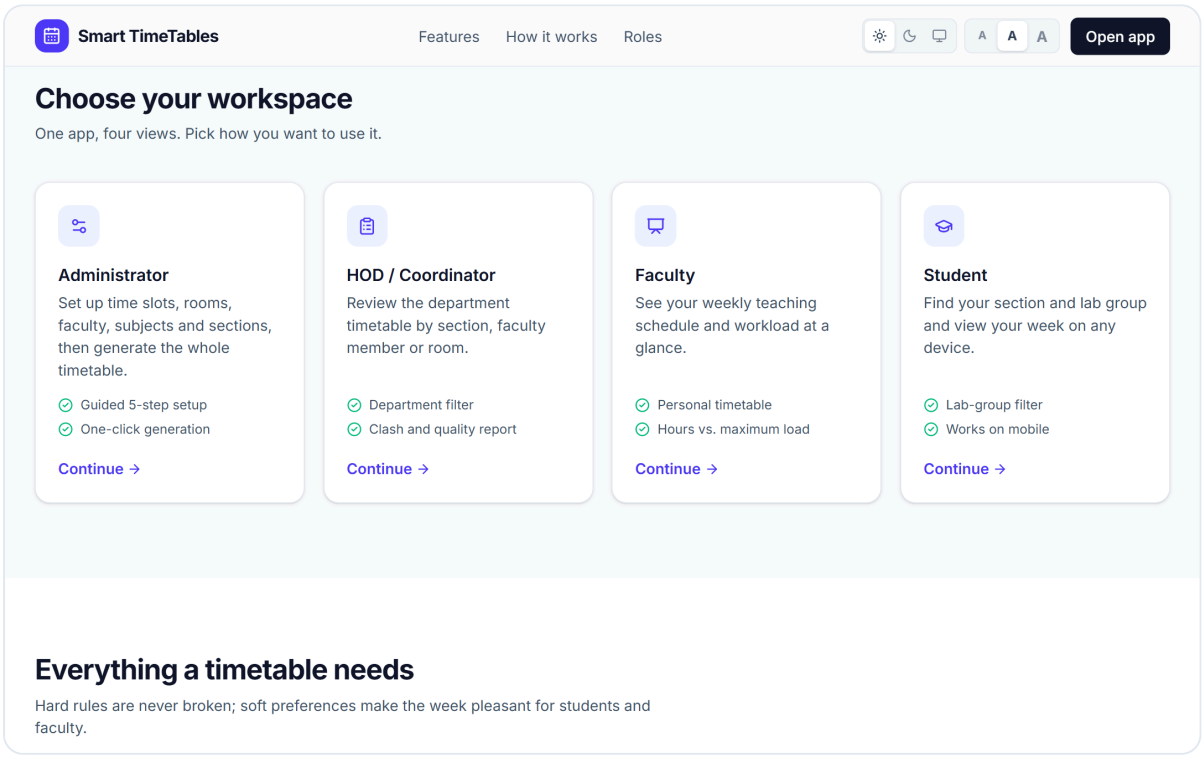


Figure 10. Landing page "Choose your workspace" section with the four roles.

08 Testing and results

Benchmark (10 runs per dataset, different random seeds)

Dataset	Classes	Complete runs	Clashes	Avg. time	Max. time	Quality (min–max, avg.)	Labs side by side
Demo college (3 depts)	114	10 / 10	0	39 ms	74 ms	95–99, avg. 96.7	n/a
GNDEC CSE (12 sections)	354	10 / 10	0	508 ms	710 ms	89–100, avg. 94.3	83%

Every run finished on the first attempt (no restarts needed).

Automated tests (18 / 18 passing)

Group	What is checked
Demo college (7 tests)	Pre-check passes; complete clash-free timetable for 5 different seeds; every section gets exactly its hours; labs are consecutive blocks in lab rooms; quality score ≥ 70
GNDEC data (3 tests)	Complete clash-free timetable for 2 seeds; every lab group gets every lab session with the right lab teacher in a big-enough lab; parallel group labs allowed, a lecture during a group lab is flagged
Pre-check and failures (6 tests)	Teacher over maximum hours; missing lab rooms; lab longer than the gap between breaks; too few lab blocks; plain-language search-failure reason; clash checker catches a double-booked teacher
AI assistant context (2 tests)	Timetable summary lists every class with time, teacher and room, is identical for identical input (needed for caching), and says when no timetable exists

Manual checks

- Every screen checked in the browser at desktop and phone widths, in light and dark themes.
- Print layout checked with a print-preview simulation: only the timetable prints, on one A4 landscape page.
- AI assistant endpoint checked end to end: input validation, missing-key message, and the request reaching the Claude API.
- Production build checked: the API key and AI library are not included in the browser code.

09 Development timeline

Built in small steps; each step was tested and committed to Git before moving on.

- **Step 1 · 14:09** Project setup: Vite, React, Tailwind CSS, Vitest and project documentation.
- **Step 2 · 14:11** Data model, demo college data, browser storage and role selection landing page.
- **Step 3 · 14:17** Backtracking scheduler with pre-check, clash verifier, quality score and tests.
- **Step 4 · 14:19** Administrator setup wizard with demo loader and Generate action.
- **Step 5 · 14:20** Colour-coded timetable grid, summary panel, and section / teacher / room views.
- **Step 6 · 14:21** Mobile-friendly student view and faculty schedule view.
- **Step 7 · 14:21** README and documentation.
- **Step 8 · 14:54** Real clock times; grid redesigned with days across and time slots down.
- **Step 9 · 14:56** Light / dark / system theme, adjustable text size, Inter font.
- **Step 10 · 15:04** Lab groups, multiple lab sessions and lab teachers; GNDEC Ludhiana CSE dataset; side-by-side lab display.
- **Step 11 · 15:25** Professional landing page, full-width layout, Claude-powered timetable assistant.
- **Step 12 · 15:45** "Smart TimeTables" branding, redesigned timetable cards and legend, faculty year filter, print support.

10 Next steps

Item	Plan
Deploy	Push to GitHub and deploy on Vercel for a public link.
Enable the AI assistant	Add the Claude API key as an environment variable (locally in `.env.local`, on Vercel in project settings) and test live answers.
More GNDEC coverage	Add D4 (7th semester), electives that combine sections, and per-year lunch slots.
Faculty preferences	Preferred days or times as soft constraints.
Manual adjustments	Drag and drop a class to another slot with live clash checking.
Import / export	Import from Excel / CSV and export timetables as Excel or PDF in bulk.

How to run

`npm install` → `npm run dev` → open `http://localhost:5173` → Administrator → **Load GNDEC CSE (real data)** → Sections → **Generate timetable**. Tests: `npm test`.